

Attention Networks: A simple way to understand Self-Attention

“Every once in a while, a revolutionary product comes along that changes everything.” — Steve Jobs

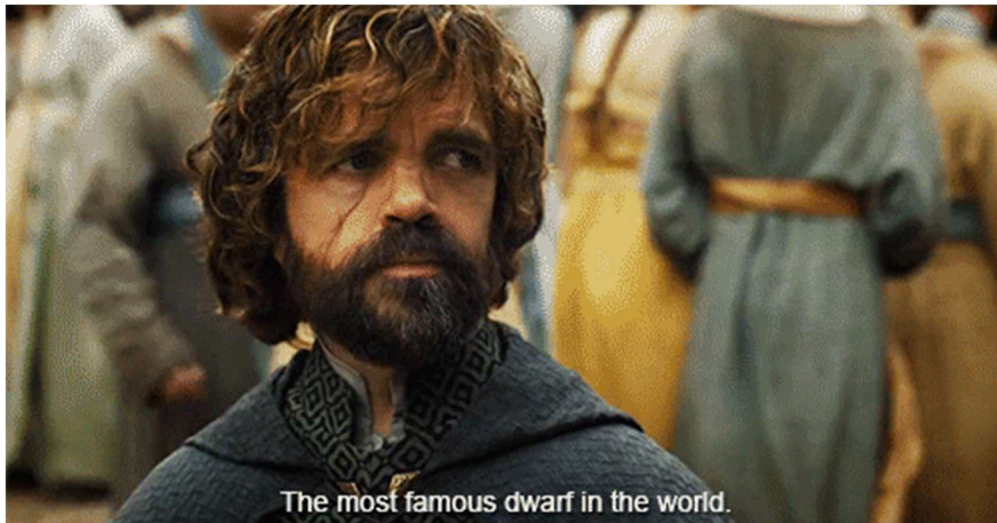
Since the paper [Attention Is All You Need](#) came out in 2017, Attention Networks have been used widely in many fields mainly in NLP and computer vision as people understood how useful they could be. But to understand them clearly the paper wasn't enough for me.



I had to go through many articles and videos to understand how they work. So in this article or maybe a series of articles (Just 2 or 3, *PROMISE!*), I will try to write down all my learnings and hope you won't have to go through many to understand attention networks and how they work.

Introduction to Attention Mechanism

To understand with an example if I had to ask you which TV series this dialogue belongs to **“A Lannister Always Pays His Debts”**? To answer this question your mind won't pay equal attention to all the words in the sentence, In fact, if you have watched the series you would know that just the word **“Lannister”** is enough to tell that this TV series is Game of Thrones and this dialogue was said by Tyrion Lannister.

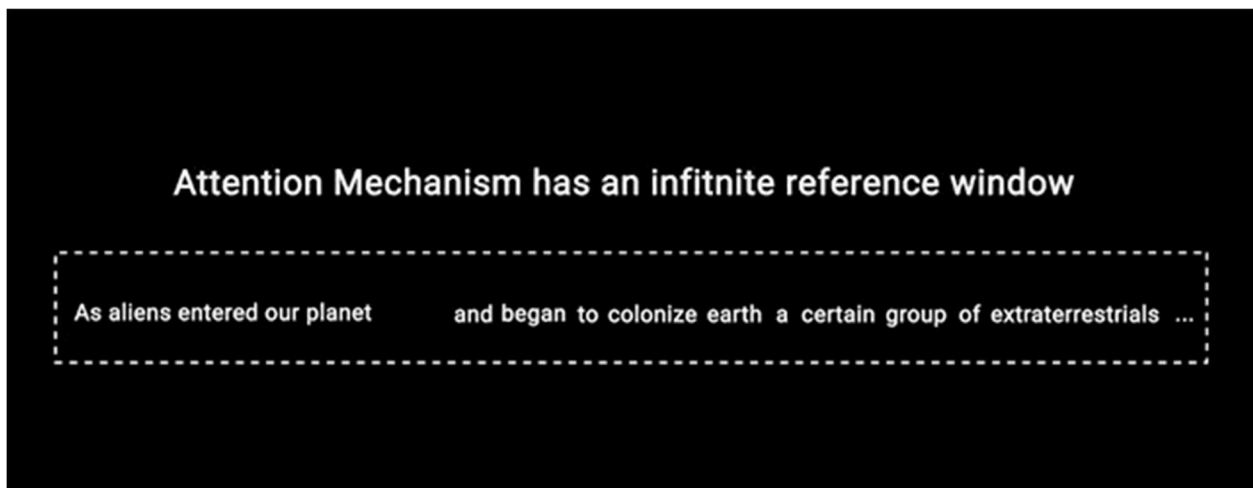


Source: Google Images

So, the Attention mechanism is based on a common-sensical intuition that we “**attend to**” a certain part when processing a large amount of information.

Why were Attention Mechanism Needed in the first place?

The Attention mechanism enables the transformers to have extremely long-term memory. A transformer model can “**attend**” or “**focus**” on all previous tokens that have been generated.

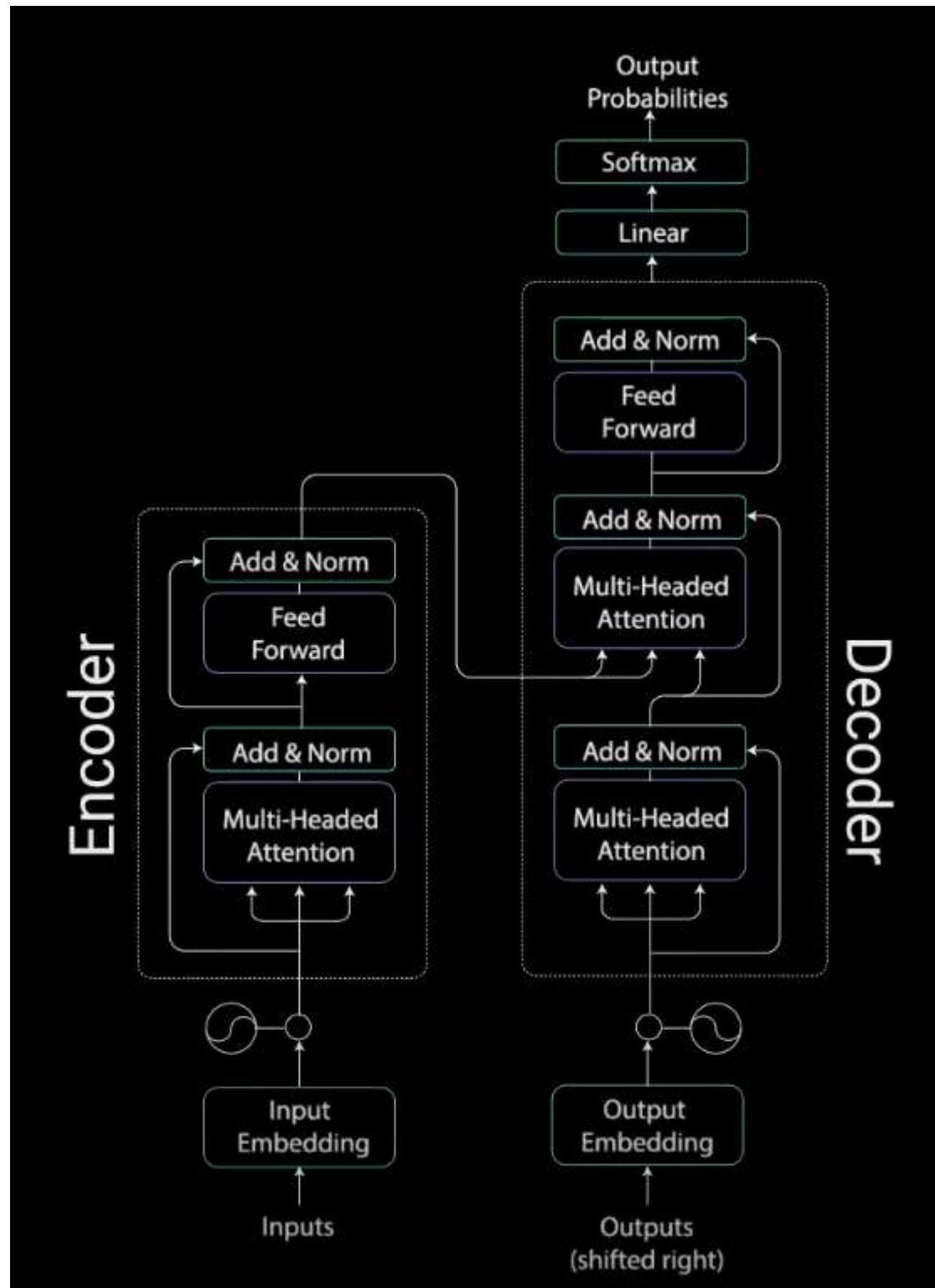


Source: Illustrated Guide to Transformers- Step by Step Explanation by [Michael Phi](#)

Recurrent neural networks (RNN) are also capable of looking at previous inputs too. But RNNs have a shorter window to reference from, so when the story gets longer, RNNs can’t access words generated earlier in the sequence. This is the same for Gated Recurrent Units (GRUs) and Long-short Term Memory (LSTM) networks. They have a longer window to reference from RNN but the attention mechanism, in theory, and given enough compute resources, has an infinite

window to reference from, therefore being capable of using the entire context of the story while generating the text.

Press enter or click to view image in full size



Source: Illustrated Guide to Transformers- Step by Step Explanation by [Michael Phi](#)

What is Self-Attention?

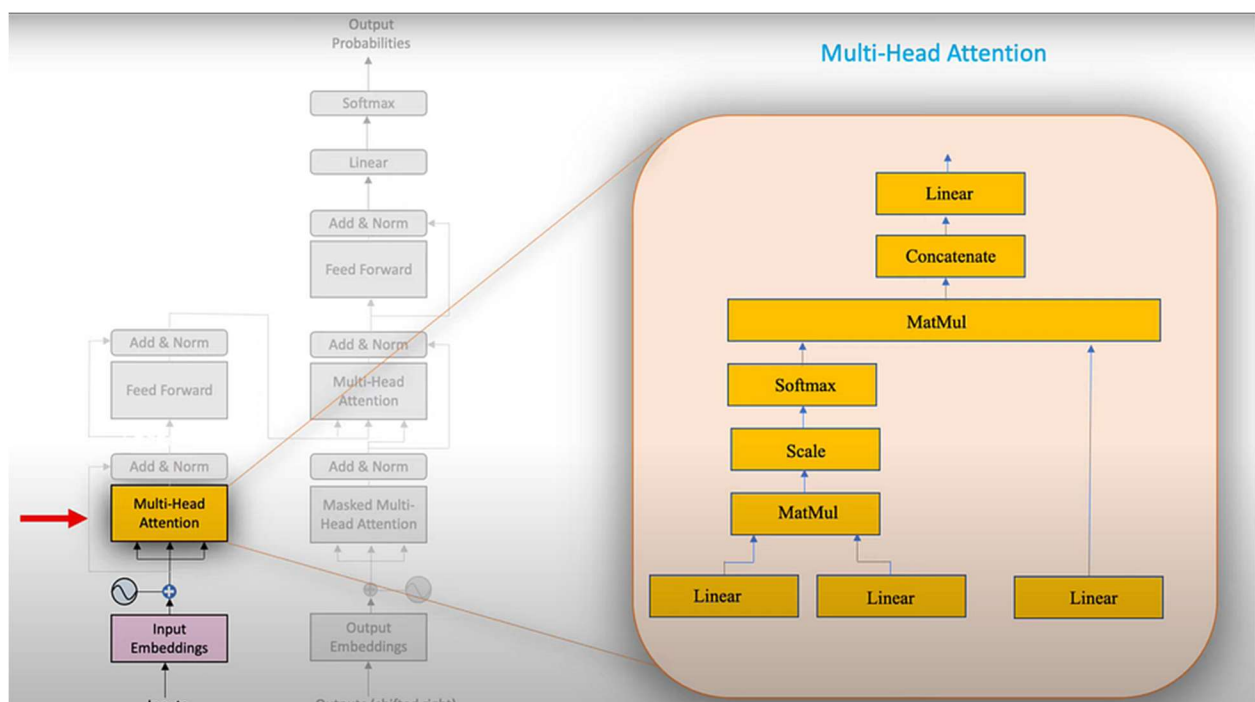
Self-Attention is when it takes into consideration the relationship among words within the same sentence. To understand this in more detail we need to understand three things:

- 1: What is QUERY, KEY & VALUE?
- 2: What is Positional Encoding?
- 3: What do we pass to QUERY, KEY & VALUE?

So, let's dig into the encoder model which the paper "**Attention is all you need**" introduces to answer all three of them.

What is QUERY, KEY and VALUE?

Press enter or click to view image in full size



Source: [Visual Guide to Transformer Neural Networks — \(Episode 2\) Multi-Head & Self-Attention](#)

The three linear layers which you see in the above image take three things as an input — "**QUERY, KEY & VALUE**". Let's take an example to understand them too.

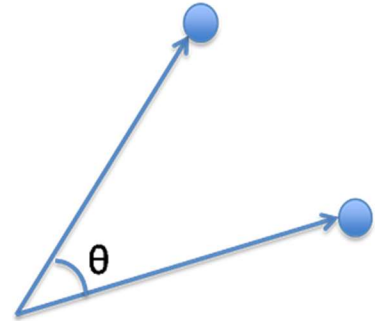
So if you were to search for something on Youtube or Google, The text which you type in the search box is the **QUERY**. The results which appear as the video or article title are the **KEY** and

the content inside them is the **VALUE**. So as to find the best matches the Query has to find the similarity between it and the Keys.

To compute the similarity between the Query and the Key, we take the help of the **Cosine Similarity** method. It is a great way to find similarities between two vectors. Cosine similarity varies from range +1 to -1, where +1 is most similar and -1 most dissimilar.

Press enter or click to view image in full size

$$\text{sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$



Source: Google Images

We can rewrite the equation like

Press enter or click to view image in full size

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\text{scaling}}$$

Since we were to multiply two matrices, we can transpose the B matrix.

$$\text{Similarity}(A, B) = \frac{A \cdot B^T}{\text{scaling}}$$

And we can be to compute the similarity between Query and Key vector matrices so we can finally write our equation like

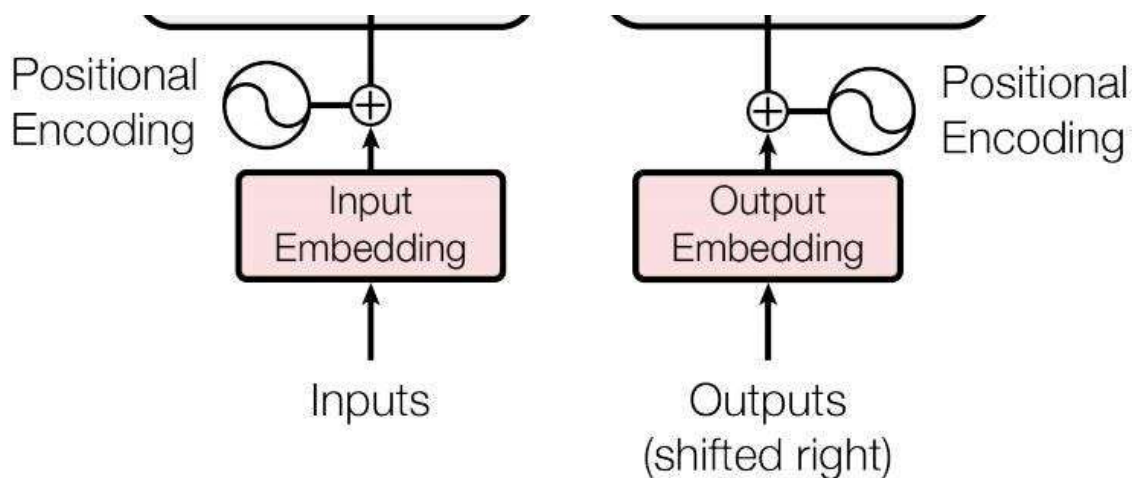
Press enter or click to view image in full size

$$\text{Similarity}(Q, K) = \frac{Q \cdot K^T}{\text{scaling}}$$

Now that we understand the concept of Similarity and what QUERY, KEY & VALUE, let's get answers to the second question. What is Positional Encoding?

What is Positional Encoding?

Press enter or click to view image in full size



Source: Google Images

As we know while dealing with textual data, we need to convert it into numbers before feeding it into any machine learning model, including neural networks. The embedding layer enables us to convert each word into a fixed-length vector of defined size. The resultant vector is a dense one with real values instead of just 0s and 1s. The fixed length of word vectors helps us to represent words in a better way along with reduced dimensions. This way embedding layer works as a lookup table. The words are the keys in this table, while the dense word vectors are the values.

The main reason we need Position Encoding is that unlike LSTM taking one input embedding at a time sequentially, Transformers take all embedding at once. Though this helps the transformer to be much faster they lose the information related to word order. To solve this problem authors of the paper ***"Attention is all you need"*** came up with a clever idea. They used wave frequencies to capture positional information.

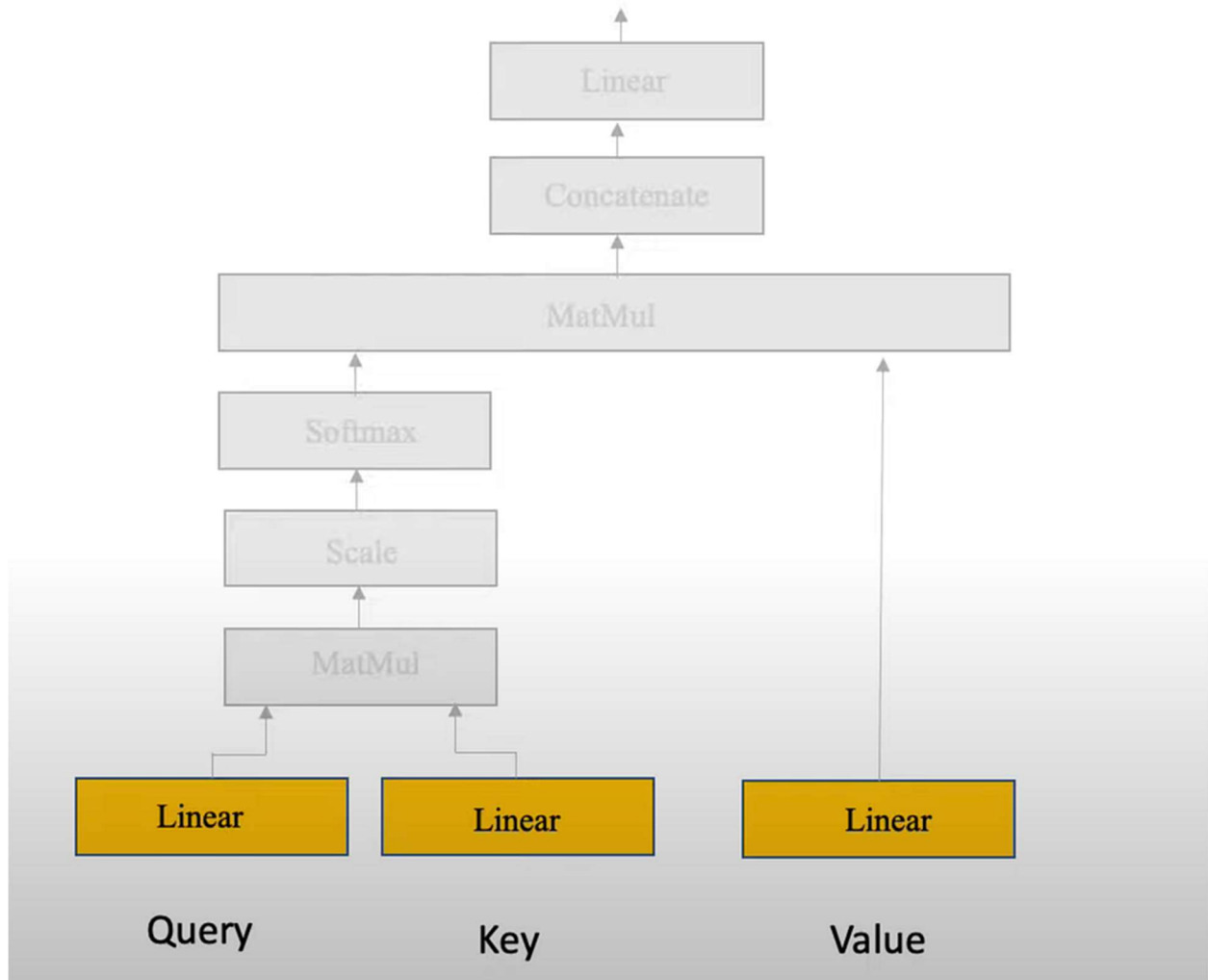
Press enter or click to view image in full size

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$

Source: [Attention is all you need paper](#)

Let's not get into much detail about this right now. We can cover this topic later in the series. If you want to read about them now, you can refer to this answer from [stack exchange](#). For now, let's just have the knowledge that positional encoding helped authors get a unique position for each word embedding.

What do we pass to QUERY, KEY and VALUE?



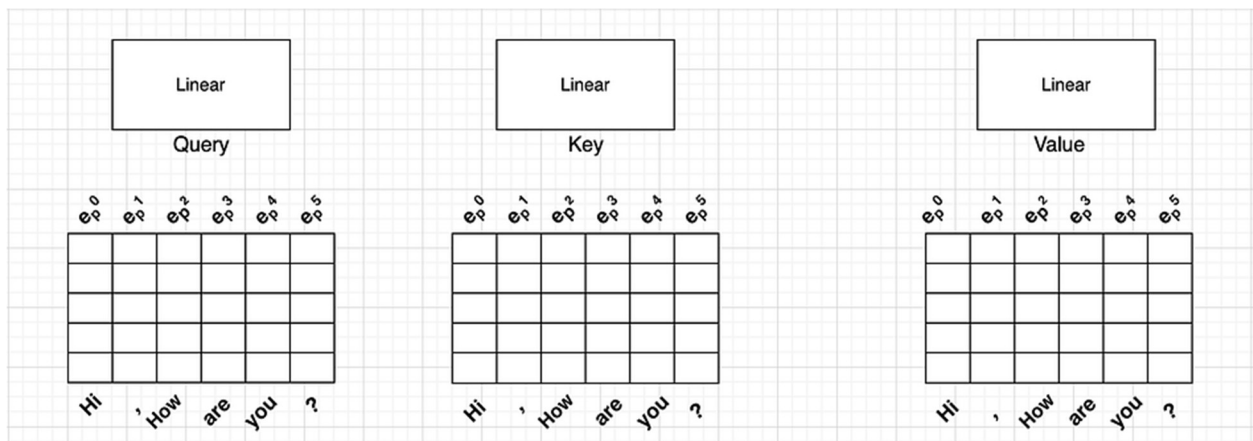
Well to the QUERY layer we feed our position-aware embeddings. We then make two more copies of that embedding and feed it to the KEY and VALUE layers too. This doesn't make any sense, why are we feeding the same embedding to all three layers, right? Well, this is a place where **SELF-ATTENTION** comes into the picture.



Source: Google Images

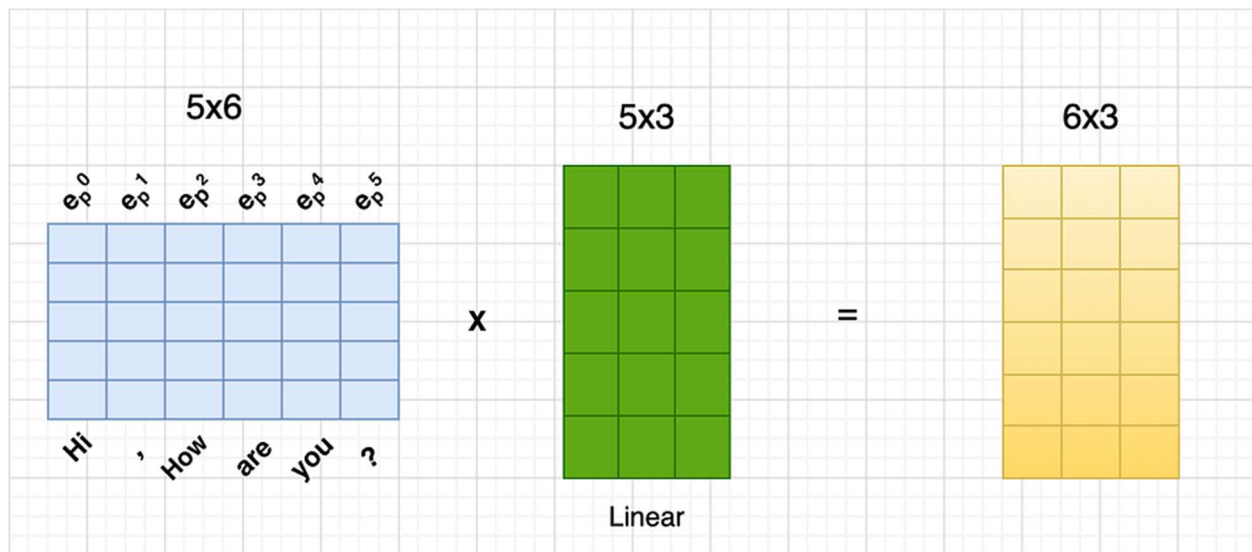
So, Let's take an example again. Let's say you want to input a sentence: *"Hi, How are you?"* and you want your transformer to output *"I am fine"*.

Press enter or click to view image in full size



In order to do this, we will pass the input sequence to our input embedding layer and then do the position encoding. Then we pass these positional aware embeddings to our linear layers.

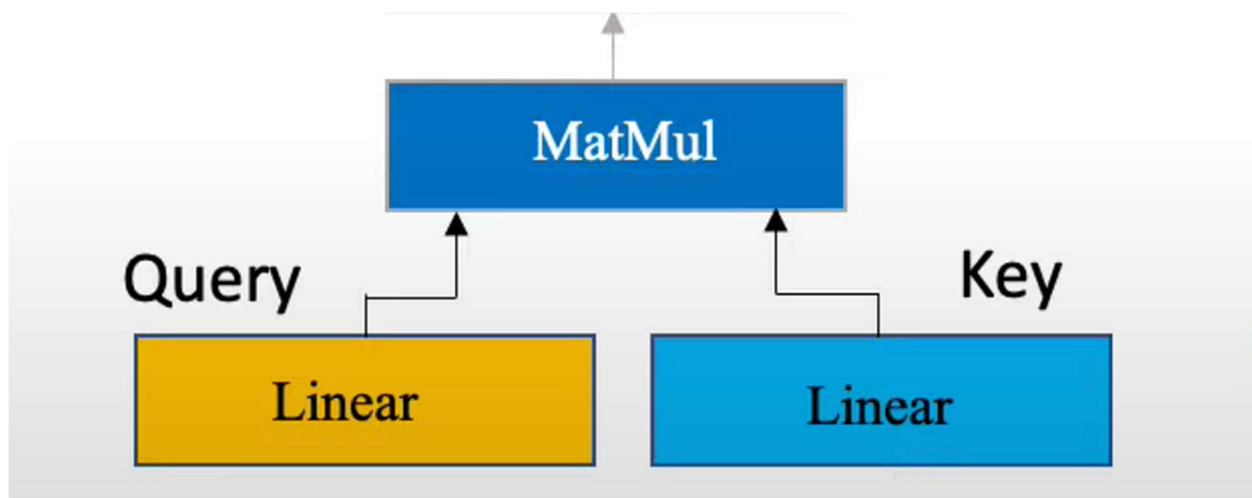
Press enter or click to view image in full size



Please note that before passing the positional aware embeddings to our linear layer we transpose them, thus making it of size (6x5). We do the same process for all three QUERY, KEY & VALUE linear layers.

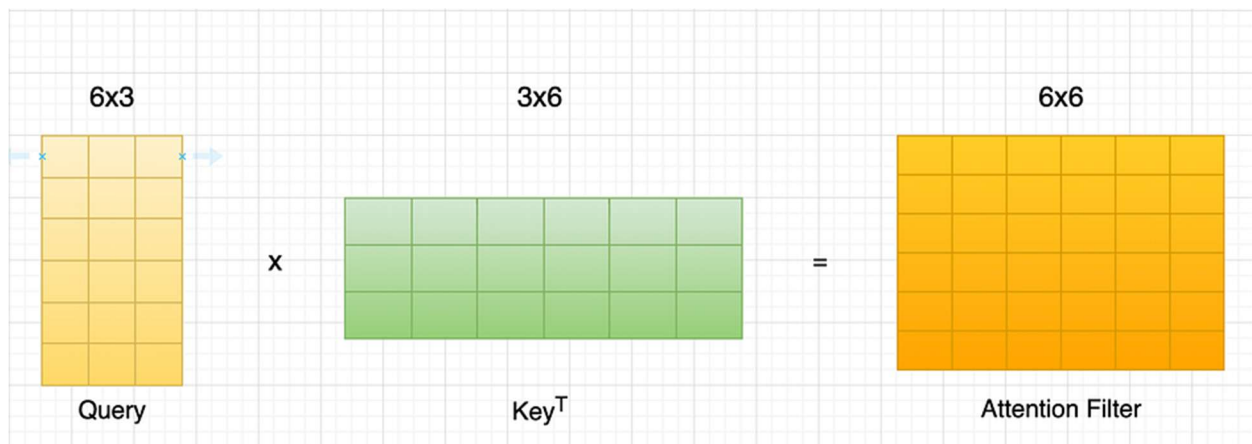
Now, let's focus on QUERY and KEY matrices if you remember we used them to find the similarities. The output of QUERY and KEY linear layers goes to the Matrix multiplication step in the network.

Press enter or click to view image in full size



The output of this dot product can be called an **Attention filter**.

Press enter or click to view image in full size



If we focus on Attention filters.

[Press enter or click to view image in full size](#)

Hi	<u>89</u>	<u>12</u>	<u>32</u>	<u>45</u>	<u>73</u>	<u>34</u>
,	<u>20</u>	<u>11</u>	<u>15</u>	<u>21</u>	<u>29</u>	<u>24</u>
How	<u>33</u>	<u>25</u>	<u>91</u>	<u>36</u>	<u>55</u>	<u>45</u>
are	<u>52</u>	<u>68</u>	<u>12</u>	<u>13</u>	<u>40</u>	<u>27</u>
you	<u>28</u>	<u>27</u>	<u>54</u>	<u>41</u>	<u>92</u>	<u>12</u>
?	<u>39</u>	<u>30</u>	<u>73</u>	<u>64</u>	<u>12</u>	<u>74</u>
	Hi	,	How	are	you	?

At first, the weights in Attention filters are more or less random numbers. But once the training process is done they take on more meaningful full values. And finally, they become Attention scores. Next, we scale these attention scores.

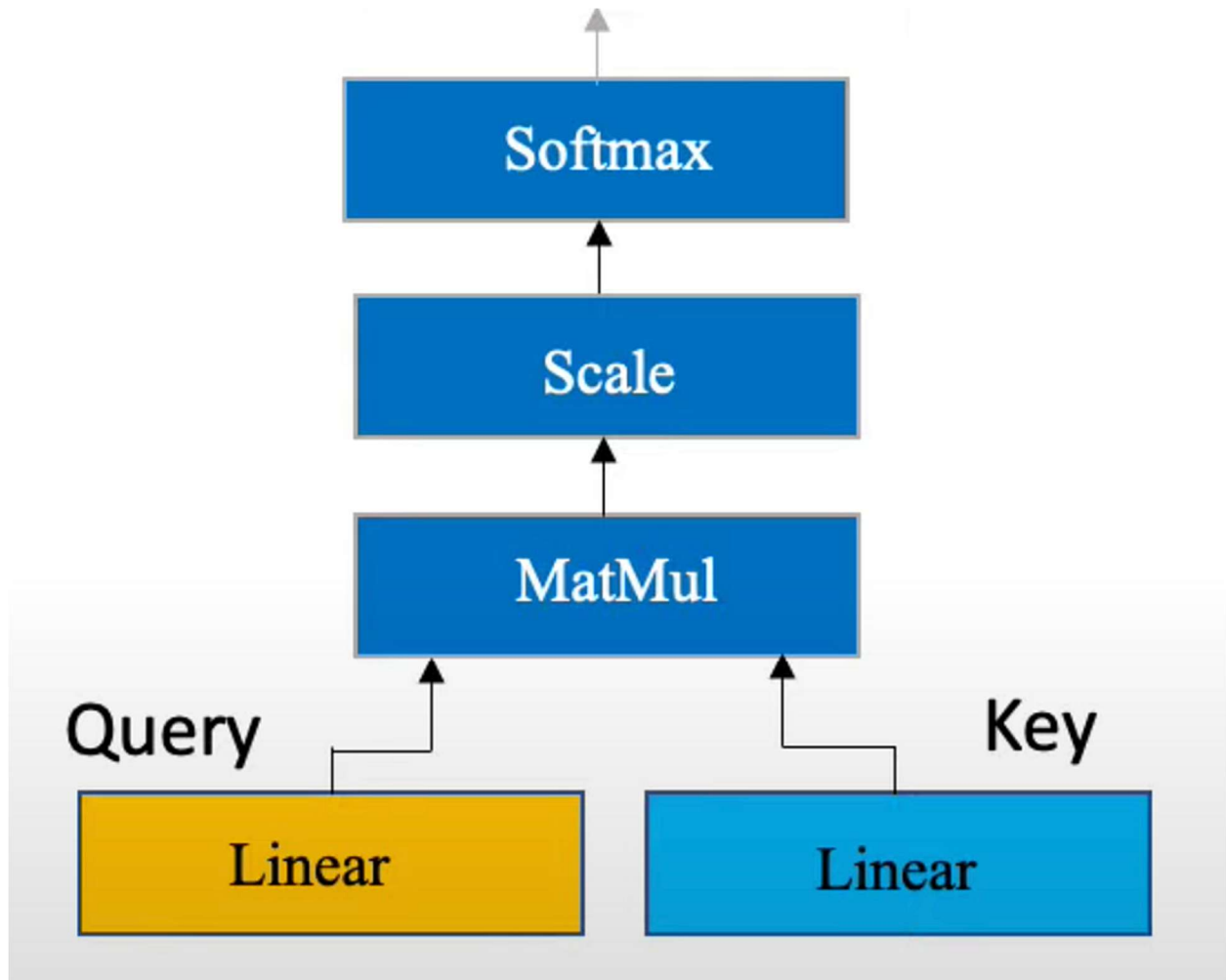
[Press enter or click to view image in full size](#)

Hi	89	12	32	45	73	34
,	20	11	15	21	29	24
How	33	25	91	36	55	45
are	52	68	12	13	40	27
you	28	27	54	41	92	12
?	39	30	73	64	12	74

$\sqrt{6}$

Finally, we squash our attention scores between the values 0 and 1 by using the softmax function.

[Press enter or click to view image in full size](#)

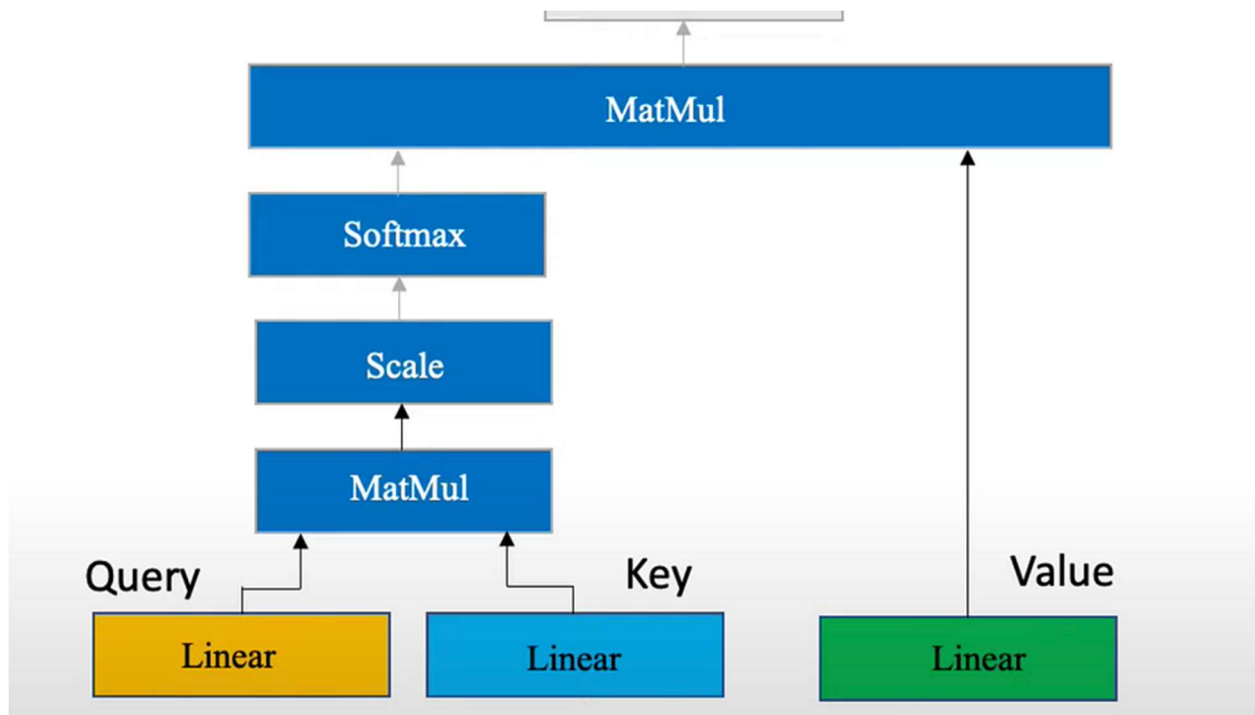


[Press enter or click to view image in full size](#)

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

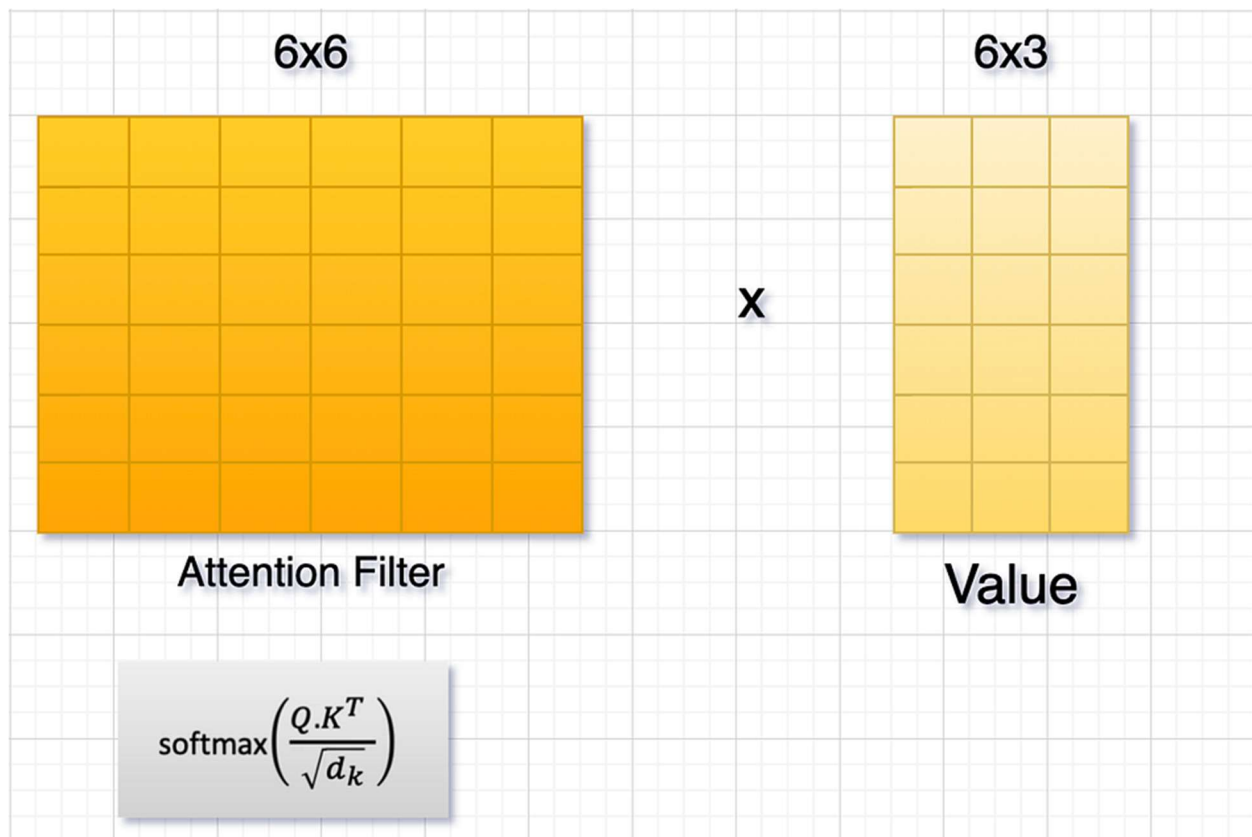
And then we get our final **Attention Filter**.

[Press enter or click to view image in full size](#)



The last main step in this network is to multiply the attention filter which we created with the value matrix that we left out at the start.

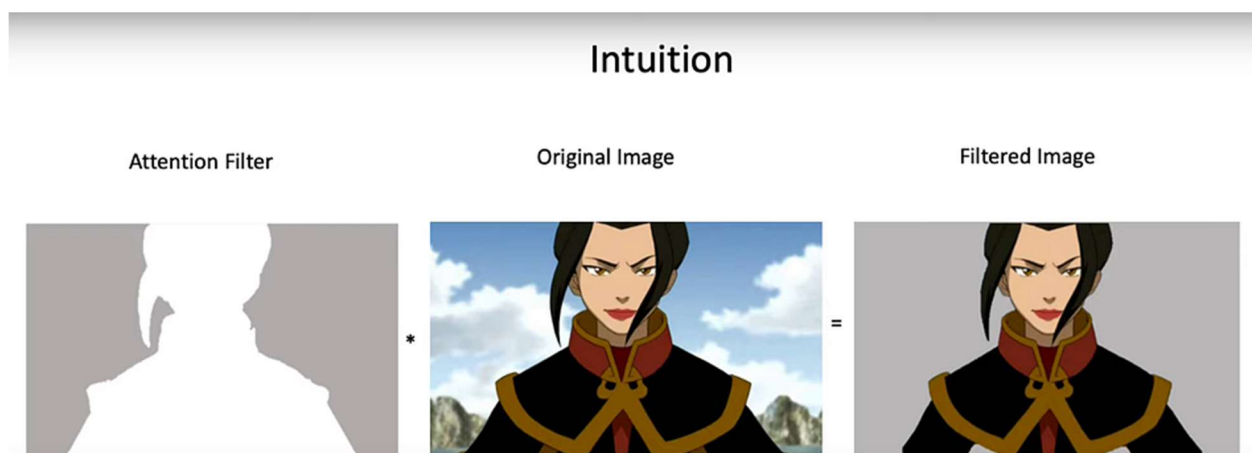
[Press enter or click to view image in full size](#)



Now if you still have the question why on earth did we go through this? What is the purpose of this attention filter? Let's take a little detour and understand this with a computer vision example.

If it was an image we were working on then when we would have multiplied the attention filter with the value matrix and all the unnecessary information would have been thrown away.

[Press enter or click to view image in full size](#)



Source: [Visual Guide to Transformer Neural Networks — \(Episode 2\) Multi-Head & Self-Attention](#)

I hope it got clear why Attention filters are necessary.

Finally, we pass the result of the **Attention filter x Value** matrix to a linear layer to get the desired output shape.

This is also called Scaled Dot Product Attention:

Press enter or click to view image in full size

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Source: [Attention is all you need](#)

So, this was it. This is what Self-Attention means and how we perform it step by step. Because of this network, both NLP and Computer Vision fields benefited, and using this network they achieved great results.

Masked Multi Head Attention in Transformer



[Sachin Soni](#)

Follow

10 min read

.

Sep 6, 2024

34

7

Transformers have become a powerful tool in tasks like language translation and text generation. While the encoder helps understand the input, the decoder is responsible for generating the output. Two key components of the decoder are multi-head attention and cross-attention, which allow it to focus on different parts of the input and the output simultaneously. In this blog, we'll break down how these attention mechanisms work, preparing you for a deeper understanding of the transformer decoder in upcoming posts.

Press enter or click to view image in full size

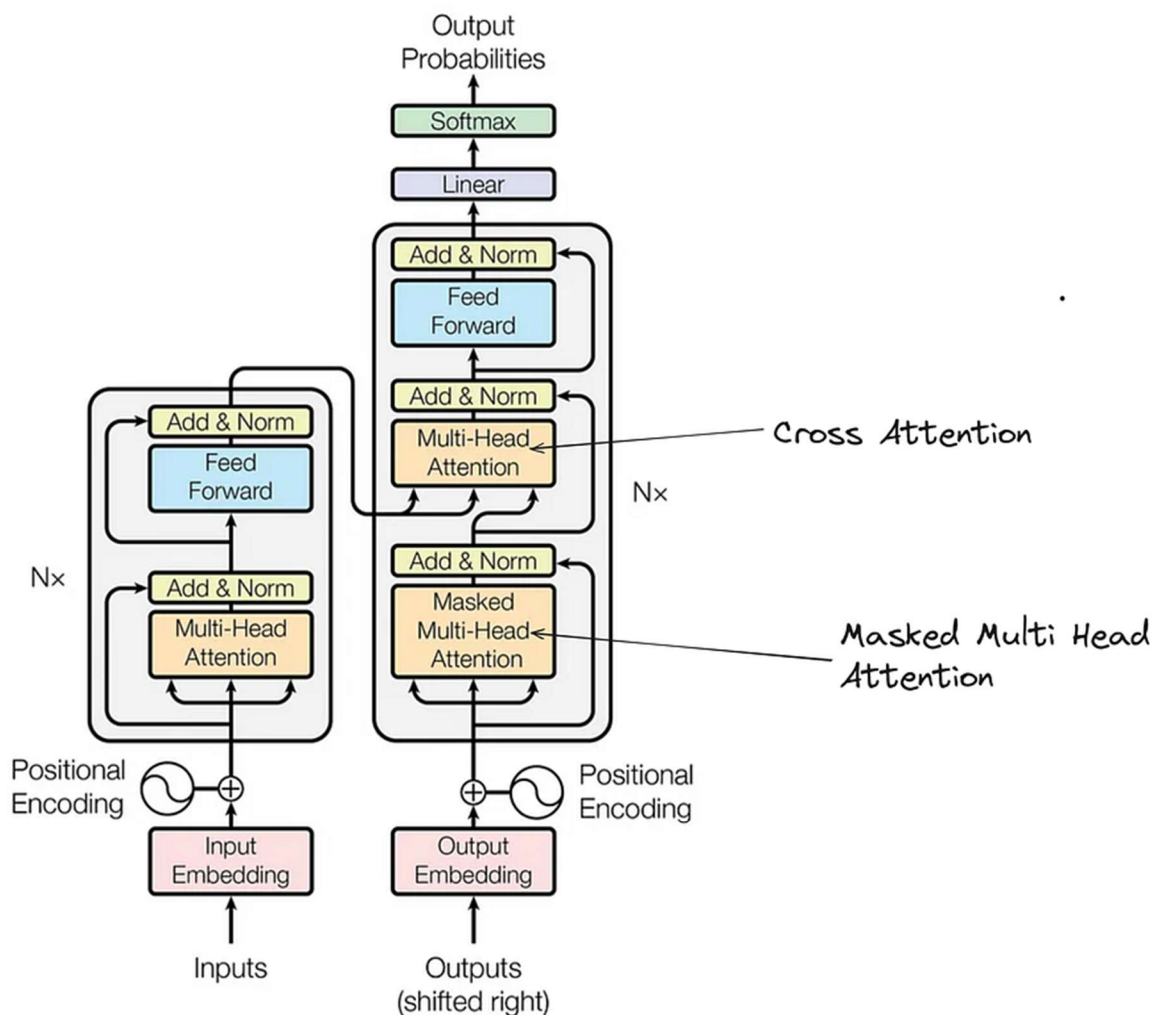


Figure 1: The Transformer - model architecture.

Before we dive into the details of Masked Head Attention, I want to extend my heartfelt gratitude to my mentor, **Nitish Sir**. His exceptional guidance and teachings on the [CampusX](#)

[YouTube channel](#) have been instrumental in shaping my understanding of this complex topic. With his support, I've embarked on this journey of exploration and learning, and I'm excited to share my insights with you all. Thank you, Nitish Sir, for being an inspiration and mentor!

The Transformer decoder is autoregressive at inference time and non-autoregressive at training time.

When we talk about the transformer decoder, it's important to know that it behaves differently during training and inference (prediction). **At inference time, the decoder works in an autoregressive manner, meaning it generates each token in a sequence by using the tokens generated so far.** In contrast, **during training, the decoder doesn't rely on its own previous outputs but uses the actual target sequence**, which allows it to be faster and more efficient.

So, what exactly is an auto-regressive model? In deep learning, auto-regressive models are those that generate data points in a sequence by conditioning each new data point on the previously generated ones. For example, let's say you're using a machine learning model to predict stock prices. To predict the price for Friday, the model will consider its predictions for Wednesday and Thursday, adjusting the prediction for Friday based on the prior information. Similarly, in a transformer decoder, each new token prediction depends on the tokens that were generated before it.

Now, you might be wondering why most sequence-to-sequence models, such as those used for tasks like language translation, text generation, or summarization, are auto-regressive. Why can't we just generate all the words at once, instead of doing it step by step?

The reason for this is simple: **when generating text, the next word often depends on the words that came before it. In tasks like language translation or text generation, each word you generate informs the next word. This is why we can't produce all the words simultaneously.** It's the sequential nature of the data that forces the model to generate outputs one at a time. Essentially, future words rely on past words, so generating them all at once would result in a loss of context and coherence. This is why models that handle such tasks need to be auto-regressive — there's simply no other way to model sequential data effectively.

The fun part here is that while the transformer decoder is auto-regressive during inference (when it's predicting or generating outputs), it behaves differently during training. During training, the decoder doesn't follow the auto-regressive approach, which may seem odd. Shouldn't the decoder act the same way during both training and inference?

The key difference lies in something called **masked self-attention**, which we'll dive into next. Understanding this concept will clarify why the transformer decoder behaves differently during training and inference, and how it helps the model efficiently learn and generate sequences.

Let's dive into the logic behind the statement that the transformer decoder behaves differently during inference compared to training. ***To prove this, we'll first start by assuming that the decoder is auto-regressive during both inference and training.***

Consider a machine translation task, where we aim to build a deep learning model to translate English sentences to Hindi using a transformer. Suppose the model has been trained, and now it's time to see how it performs during inference.

For example, let's take the English sentence: **"I am fine"**. When this sentence is fed into the model during inference, the transformer's encoder processes the entire input sentence in parallel, thanks to the self-attention mechanism. Each word — **"I"**, **"am"**, and **"fine"** — is processed simultaneously to generate respective embeddings. These embeddings undergo various layers such as multi-head attention, normalization, and feed-forward neural networks. After passing through these layers, we get final vectors for each word.

Now, these word vectors are sent to the transformer decoder. Here's where the auto-regressive nature comes into play. The decoder generates the output one word at a time. It starts by predicting the first word based on a start token and the encoded input. Suppose the model predicts the word **"मैं"** (which means "I" in Hindi) as the first word.

At this point, the decoder takes its first prediction, **"मैं"**, as an input along with the encoder's output to predict the next word. This process continues word by word, with each subsequent word depending on the previously generated word.

However, mistakes can happen. Let's assume at some point the model makes an incorrect prediction. Instead of predicting **"बढ़िया"** (which means "fine"), it predicts **"घटिया"** (which means "bad"). Due to the auto-regressive nature, this incorrect prediction is then fed into the next step, and the decoder continues with this error. By the end, the model outputs: **"मैं घटिया हूँ"** (meaning "I am bad") instead of **"मैं बढ़िया हूँ"** ("I am fine").

Press enter or click to view image in full size

Inference

Query Sentence -> I am fine

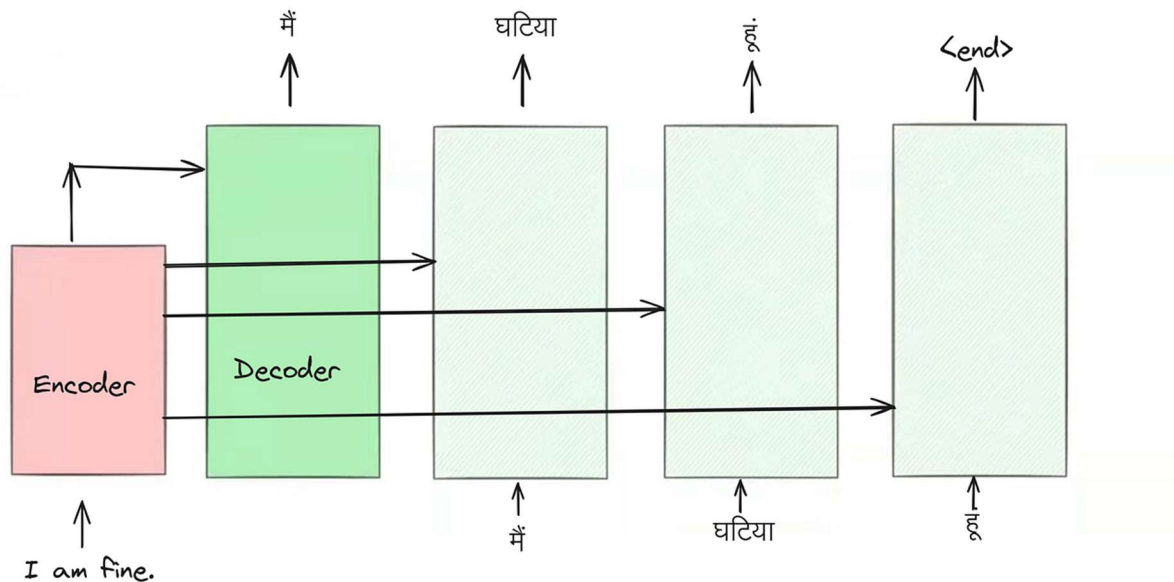


image by CampusX

This demonstrates how auto-regression works during inference. Each step relies on the previous output, making it crucial that earlier predictions are correct to avoid cascading errors.

let's step back and dive into the **training process** of a transformer model. We've already discussed inference, but training follows a slightly different approach. To understand it better, let's use the same example: translating the sentence "How are you?" from English to Hindi, where the translation would be "आप कैसे हैं".

Training a Transformer: The Auto-Regressive Approach

When we train a transformer model, we start by passing our input sentence to the encoder. In this case, "How" goes into one position, "are" in the next, and "you" in the last position. The encoder processes these words and provides an output, which will be passed on to the decoder.

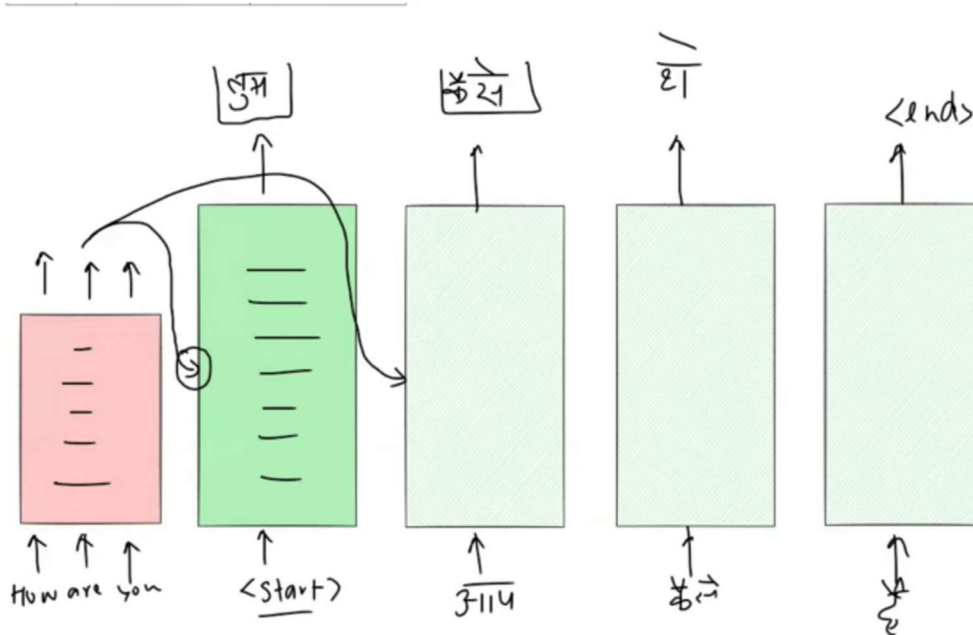
Now, here's where the interesting part begins: **auto-regressive training**. Just like in inference, the decoder in training also works sequentially. Initially, we provide a start token (let's say **<start>**) to signal the decoder to begin. Based on the input from the encoder and the start token, the decoder predicts the first word of the output sentence, which in this case should be "आप". However, if the model incorrectly predicts "तुम" instead, it's no big deal at this point.

We don't stop here. We move to the next step. But there's a twist.

Introducing Teacher Forcing

In the next time step, instead of feeding the decoder with the wrong prediction (“तुम”), we use the correct word from our dataset, “आप”. This technique is called **teacher forcing**. Even if the model makes a mistake, we feed the correct word in the next step during training.

Press enter or click to view image in full size



The decoder continues, and we keep using the correct outputs at each time step. This ensures that the training process is smoother and doesn't get derailed by incorrect predictions early on.

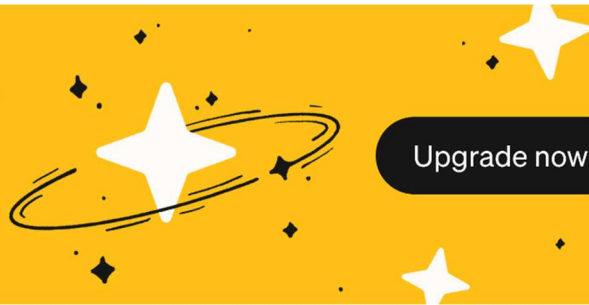
Now, why does this matter? Because **teacher forcing** allows us to train more efficiently.

The Problem with Sequential Training

If we were to treat the training process as fully auto-regressive, similar to inference, it would slow down the entire process. Think about it: every word in the sentence would have to be predicted one after the other. In the case of a small sentence like “How are you?”, we would need to pass through the decoder three times. But imagine a full paragraph with 300 words — the decoder would need to process each word individually, multiplying the time and computation required.

One subscription. Endless stories.

Become a Medium member
for unlimited reading.



For a large dataset with thousands of rows, this sequential approach would make training painfully slow.

Parallelizing the Training Process

Here's the magic of teacher forcing: during training, we already know the correct output at each time step from the dataset. So, instead of predicting each word one by one, we can **parallelize the entire process**. The correct words from the dataset can be fed into the decoder all at once, rather than waiting for the previous step's output. This eliminates the need for sequential execution in training.

In contrast, during inference, we don't have the luxury of knowing the correct output beforehand, so the model must predict one token at a time. But in training, this constraint doesn't exist, thanks to teacher forcing.

Why Training Doesn't Need to Be Auto-Regressive

During inference, the prediction of each word depends on the previous word. For example, when predicting the second word, the model relies on the output of the first. But in training, this isn't the case. The next word in the sequence is always available from the dataset, allowing us to skip the sequential nature.

Thus, we can execute multiple steps of the decoder simultaneously, significantly speeding up the training process. This is the **core advantage** of using teacher forcing.

The Real Problem with Parallel Processing :

Now, here's the fundamental problem: when you're parallelizing, you're sending all the words to the self-attention block **simultaneously**. While this works fine for generating contextual embeddings during training, there's a **serious issue**: self-attention uses information from **future tokens**, which is unfair during inference.

Let's break it down with a concrete example. At the start of our sentence, when we only have "आप", the model should not have access to "कैसे" or "हैं" because they haven't been generated

yet. However, when training in parallel, self-attention does access this future information — using “कैसे” and “हैं” to influence the embedding of “आप”.

This is essentially cheating. During training, it’s possible to use future tokens because the entire sentence is available. But during inference (or real-world predictions), we don’t have access to future tokens. The model would have just predicted “आप”, but it doesn’t know what the next word will be. Therefore, using future tokens during training leads to a mismatch between training and inference behavior, creating a phenomenon known as **data leakage**.

The Dangers of Data Leakage :

When your model accesses future information during training, it learns based on insights that won’t be available during inference. This results in a model that performs well on the training data (where future words are available) but struggles in real-world scenarios where it must predict each word step-by-step.

This is a classic example of **data leakage** — a situation where the model has access to extra information during training that it wouldn’t have during inference. Consequently, your model might perform exceedingly well on the training data but fail to generalize when making predictions on new, unseen data.

In autoregressive models, training is slow because tokens are processed one by one, but there’s no data leakage since future tokens aren’t visible. To speed things up, non-autoregressive models process tokens in parallel, but this causes data leakage, as current tokens can see future ones, which won’t happen during real predictions. This creates a dilemma: autoregressive models are accurate but slow, while non-autoregressive models are fast but prone to mistakes. The solution to this problem lies in revisiting the calculations in self-attention, where we can find a way to balance speed and accuracy.

Solution of Data Leakage : Idea of Masked Multi Head Attention :

To solve the issue of balancing speed and accuracy in transformer training, we need to dive into the details of self-attention. The key lies in the way self-attention processes input sequences. Let’s take an example sentence like “आप कैसे हैं.” First, we calculate the embeddings for each word and pass them into the self-attention block. Within this block, each word’s embedding interacts with three matrices: the query (Q), key (K), and value (V) matrices. By performing dot products with these matrices, we generate three new vectors for each word — query, key, and value vectors.

Press enter or click to view image in full size

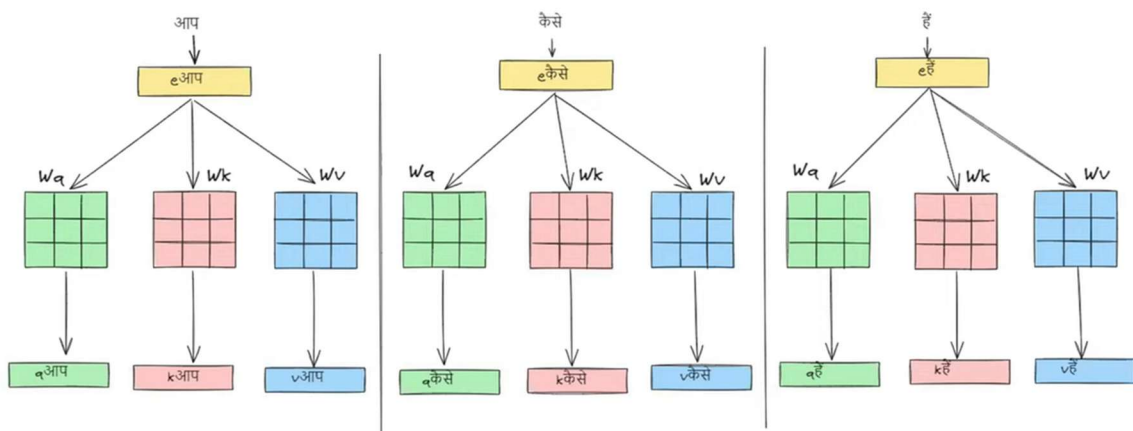


image by CampusX

Next, we stack the query vectors to form a query matrix, the key vectors to form a key matrix, and the value vectors to form a value matrix. This allows us to perform matrix operations efficiently.

[Press enter or click to view image in full size](#)

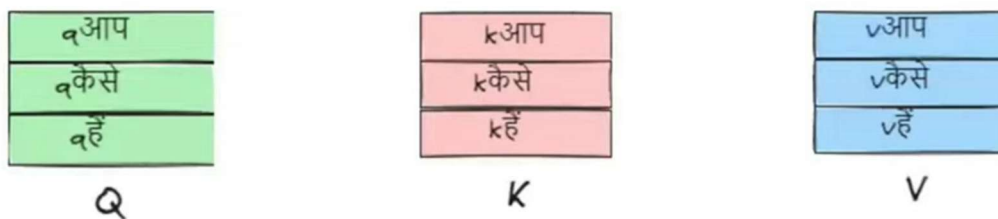


image by CampusX

The self-attention process begins by taking the dot product between the query and key matrices, resulting in a new matrix representing the attention scores. These scores are scaled and normalized using softmax to ensure they add up to one. Then, the attention scores are multiplied by the value matrix to compute the contextual embeddings for each word.

[Press enter or click to view image in full size](#)

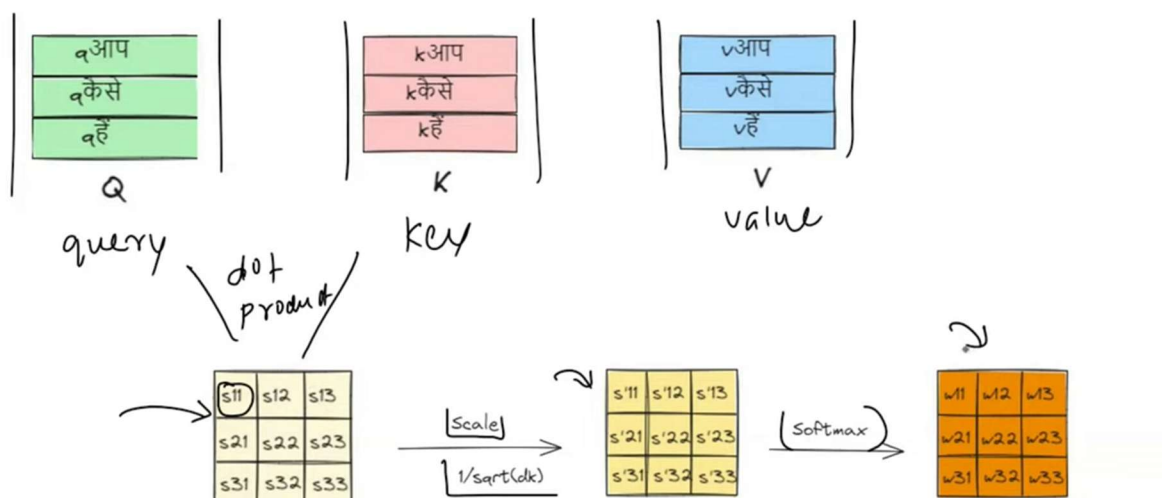


image by CampusX

Press enter or click to view image in full size

$$\begin{aligned}
 \text{आप}_{ce} &= w_{11} * \text{आप} + w_{12} * \text{कैसे} + w_{13} * \text{हैं} \\
 \text{कैसे}_{ce} &= w_{21} * \text{आप} + w_{22} * \text{कैसे} + w_{23} * \text{हैं} \\
 \text{हैं}_{ce} &= w_{31} * \text{आप} + w_{32} * \text{कैसे} + w_{33} * \text{हैं}
 \end{aligned}$$

In this diagram, we illustrate how each word relates to the others mathematically. For example, since 'आप' appears first, we mask the details of 'कैसे' and 'हैं' as they are future tokens. Similarly, since 'कैसे' comes after 'आप' but before 'हैं', we mask 'हैं' as it is a future token for the word 'कैसे'. (See below)

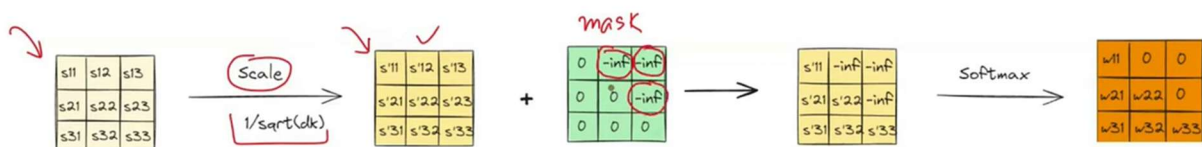
Press enter or click to view image in full size

$$\begin{aligned}
 \text{आप}_{ce} &= w_{11} * \text{✓आप} + w_{12} * \text{✓कैसे} + w_{13} * \text{✓हैं} \\
 \text{कैसे}_{ce} &= w_{21} * \text{✓आप} + w_{22} * \text{✓कैसे} + w_{23} * \text{✓हैं} \\
 \text{हैं}_{ce} &= w_{31} * \text{✓आप} + w_{32} * \text{✓कैसे} + w_{33} * \text{✓हैं}
 \end{aligned}$$

image by CampuX

However, to prevent future tokens from influencing current tokens (data leakage), we use a technique called “masking.” We introduce a mask matrix where values corresponding to unwanted future token contributions are set to negative infinity.

Press enter or click to view image in full size



Before applying the softmax function to the scaled dot product matrix, we introduce a mask that assigns values of 0 to others positions and negative infinity to positions where we want to hide future tokens during training. (See red circles in the diagram)

When applying the softmax function, these values become zero, effectively preventing any leakage from future tokens during training.

This masking process allows us to combine the benefits of both autoregressive and non-autoregressive models. We can train in parallel for efficiency, while ensuring that current tokens only attend to past tokens, thereby avoiding data leakage. This solution strikes a balance between faster training and maintaining the integrity of token prediction.